

RAG-põhise tehisaru loomine

10.03.2026



Mattias Arro

- Üle 20-aastase IT-alase kogemusega
- 10-aastase AI arenduse kogemusega
- Ehitanud 10+ AI-põhist rakendust
- AI/ML arhitekt PragmatiqAI-s
- BSc Computer Science
- MSc Data Science

Millega Pragmatiq AI tegeleb

Pragmatiq AI toetab ettevõtteid kogu digitaliseerimise teekonna vältel – pakkudes **strateegilist nõustamist**, **praktilisi koolitusi** ja **tehnilist tuge** AI-lahenduste juurutamisel, et saavutada reaalset, nähtavat arengu.

Meie töötoa eemärk #1

Tagada selge ja praktiline arusaam sellest,
kuidas töötab RAG, kuidas RAG-põhist süsteemi üles
seada ja hallata.

Meie töötoa eesmärk #2

Koostada **nimekiri** näidisküsimustest, millele soovime RAG süsteemiga vastust ning **luua selgus RAG süsteemi juurutamise järgmiste sammude osas.**

Kuidas me seda saavutame? (Agenda)

- Sissejuhatus: keelemudelid ja RAG süsteemid
- RAG süsteemi ülesseadmine on-premise keskkonnas
- RAG süsteemi opereerimine ja haldamine

12-12:45 - Lõuna

- Baasmudelite valik ja riistvara planeerimine
- Andmeallikate integreerimine: SharePointi näide
- Alternatiivlahendused ja edasised sammud

Miks AI just praegu läbi murrab?

- 2017 – Transformer arhitektuur *("Attention is All You Need")*
- 2020 – GPT-3: 175 miljardit parameetrit, üldine keeleoskus
- 2022 – ChatGPT: AI muutub kättesaadavaks kõigile
- 2023–2024 – avatud mudelid jõuavad kommertslahenduste tasemele
- 2024–2025 – ettevõttesisesed on-premise lahendused muutuvad reaalseks

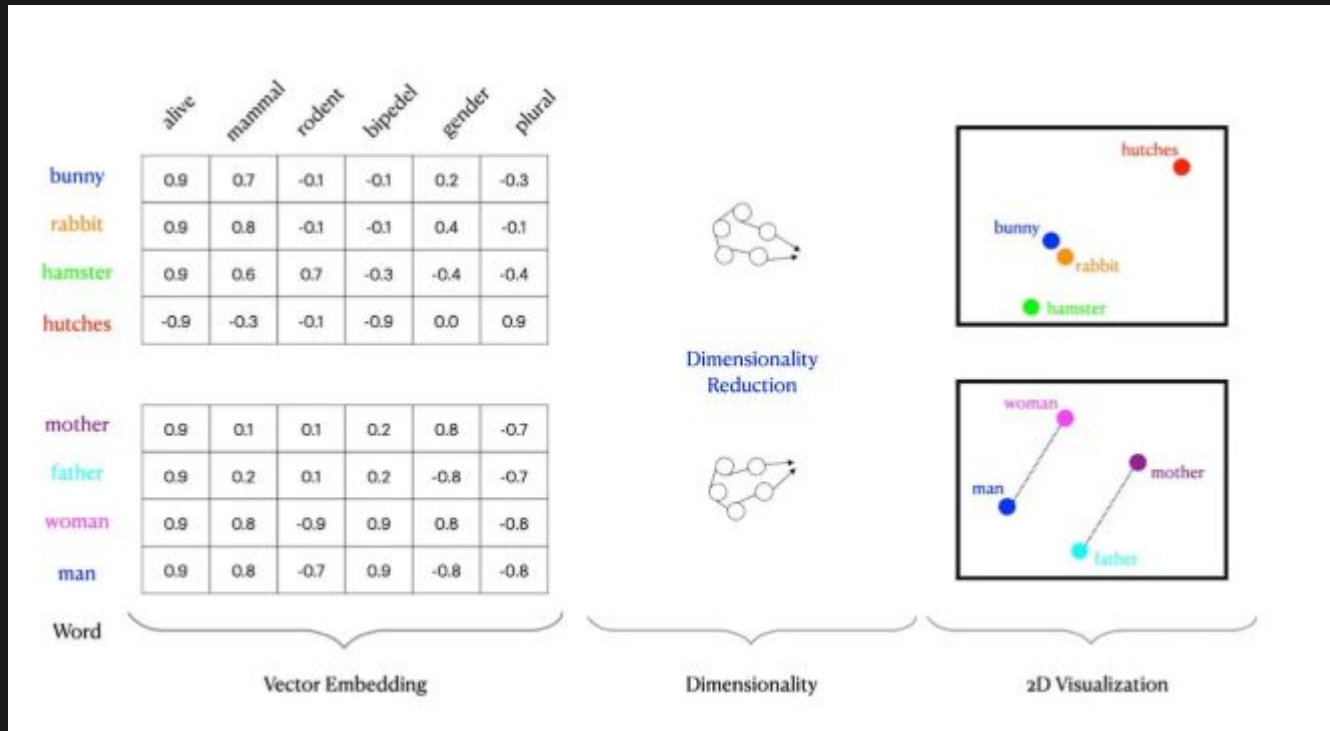
Mida keelemudelitega saab teha?

- 📄 Dokumentide analüüs – lepingute läbivaatus, kokkuvõtted, võtmepunktide eraldamine
- 🔍 **Semantiline otsing ja küsimustele vastamine – leia vastus küsimusele tuhandete dokumentide hulgast**
- ✍️ Teksti genereerimine – aruanded, e-kirjad, tõlked, koodikommentaariid
- 💬 Vestlusrobotid – klienditugi, siseabi, IT helpdesk
- 📅 Struktureeritud andmete eraldamine – täida vorm vabatekstist, parsi arved
- 🌐 Tõlkimine – dokumentide tõlge, sh erialakeelega
- 💻 Koodiabi – koodi kirjutamine, selgitamine, silumine
- 📁 Klassifitseerimine ja märgendamine – kategoriseeri dokumendid, tuvasta teema ja meelsus
- 📊 Andmete tõlgendamine – selgita tabeleid ja graafikuid loomulikus keeles
- 🔔 Anomaaliade tuvastamine – logi- ja andmevoogude jälgimine, erandite selgitamine

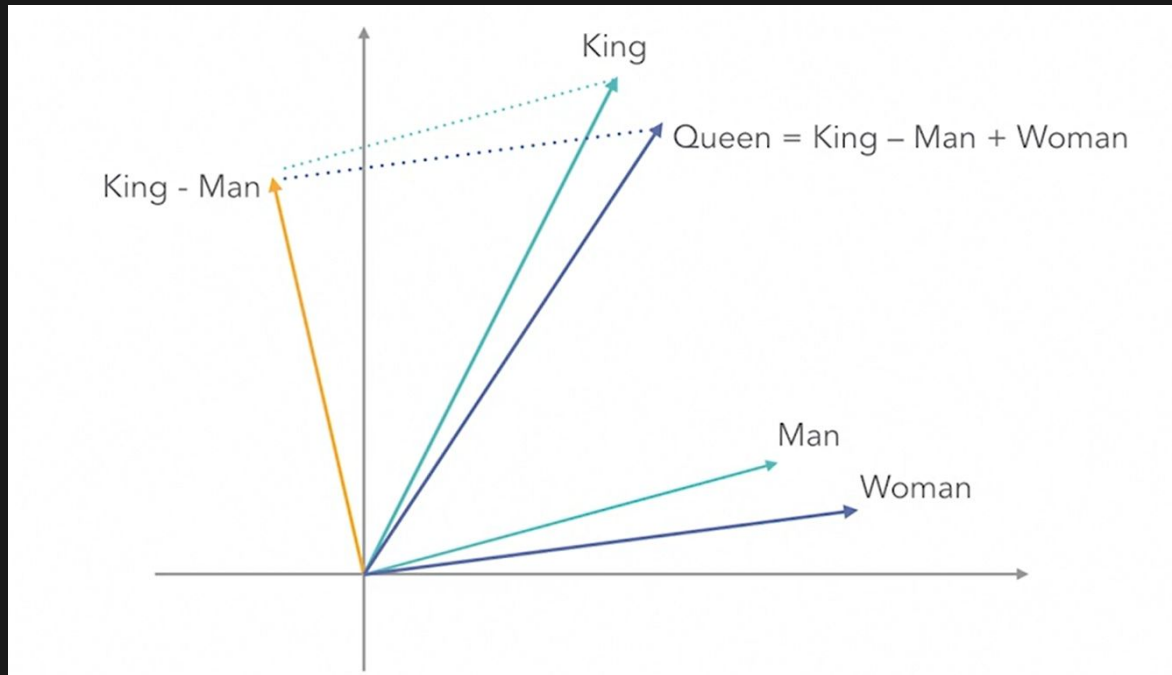
Keelemudelid – põhiprintsiibid

- **Mis on token?**
 - Sõna osa, sõna või kirjavahemärk
 - "tehisintellekt" → ["tehis", "intel", "lekt"]
- **Kuidas mudel teksti genereerib?**
 - Sisend: "Pealinn on ____"
 - Mudel: arvutab tõenäosused kõigi sõnade jaoks
 - Tulemus: "Tallinn" (tõenäosus 94%)
 - "linn" (tõenäosus 3%)
 - ...
- **Kontekstiaken (context window)**
 - Mudel "näeb" korraga piiratud hulga teksti (nt 8 000 – 128 000 tokenit)
 - Kõik, mida mudel teab, peab olema selles aknas
 - ⚠ Mudel ei mäleta eelmisi vestlusi – iga päring algab nullist

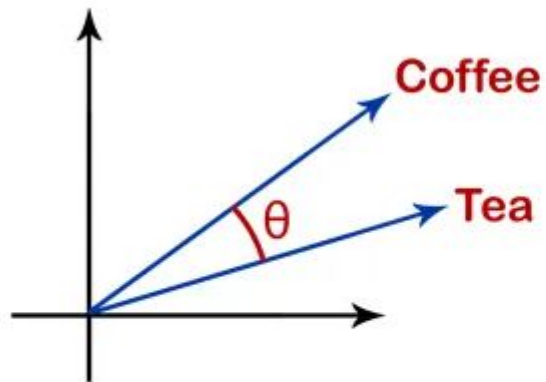
Taust – vektorid ja tehted vektoritega



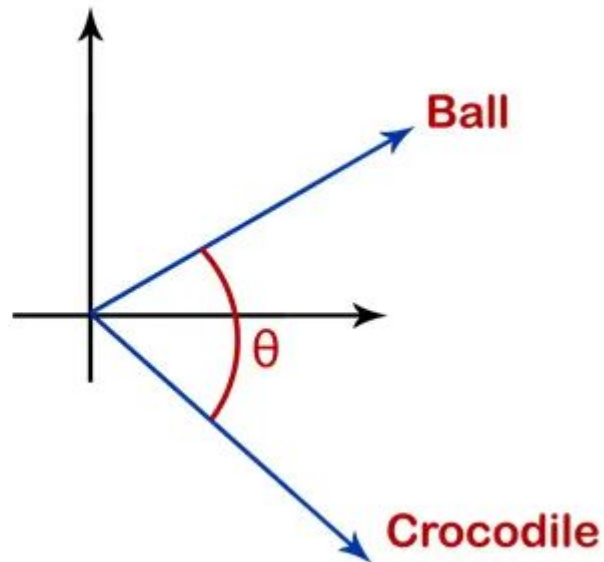
Taust – vektorid ja tehted vektoritega



Taust – vektorid ja tehted vektoritega



$$\text{sim}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$



Vector store – vektorite andmebaas

Tavaline andmebaas vs vector store

	Tavaline andmebaas	Vector store
Salvestab	Tekst, numbrid, kuupäevad	Arvude vektorid
Otsib	Täpse vastega (WHERE keel = 'eesti')	Sarnasuse järgi (lähim vektor)
Kasutusjuht	Kasutajate andmed, tehingud	Semantiline dokumentide otsing

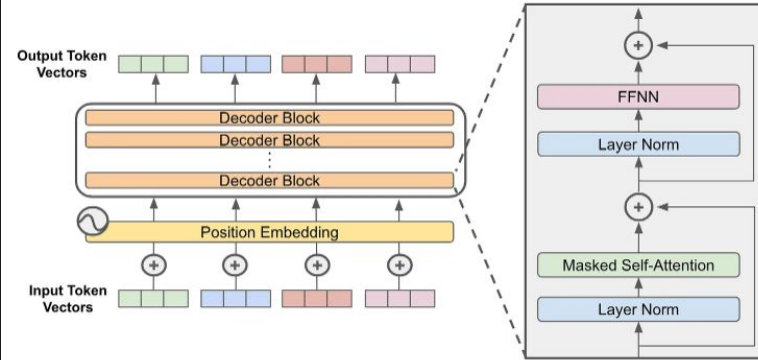
Populaarsed vector store lahendused

- Qdrant – hea Docker tugi, kirjutatud Rustis
- **Chroma – lihtne seadistada, hea arenduseks**
- Weaviate – täisfunktsionaalne, sisseehitatud otsinguserver
- pgvector – PostgreSQL laiendus, tuttav infrastruktuur

LLM & VLM

- Mudeli arhitektuur
 - *Input embedding*
 - *Positional embedding*
 - *Decoder block*
 - *Output embedding*
 - *Classification head*
- Treenimise faasid
 - *Pre-training: next token prediction*
 - *Alignment: SFT & RLHF*
 - *Fine-tuning*

The Full Decoder-Only Transformer Model



Structure of a decoder-only transformer model

Miks ei piisa ainult keel mudelist?

- **Probleem 1: Hallutsinatsioonid**
 - Mudel genereerib usutava kuid vale vastuse – ta ei tea, mida ta ei tea
- **Probleem 2: Aegunud info**
 - Mudel on treenitud ajaloolistel andmetel – ta ei tea tänasest muutustest
- **Probleem 3: Konfidentsiaalne info**
 - Teie siseandmeid pole mudelile treenimiseks antud
- **Probleem 4: Allikate puudumine**
 - Mudel ei suuda viidata konkreetsele dokumendile, mis vastust põhjendaks

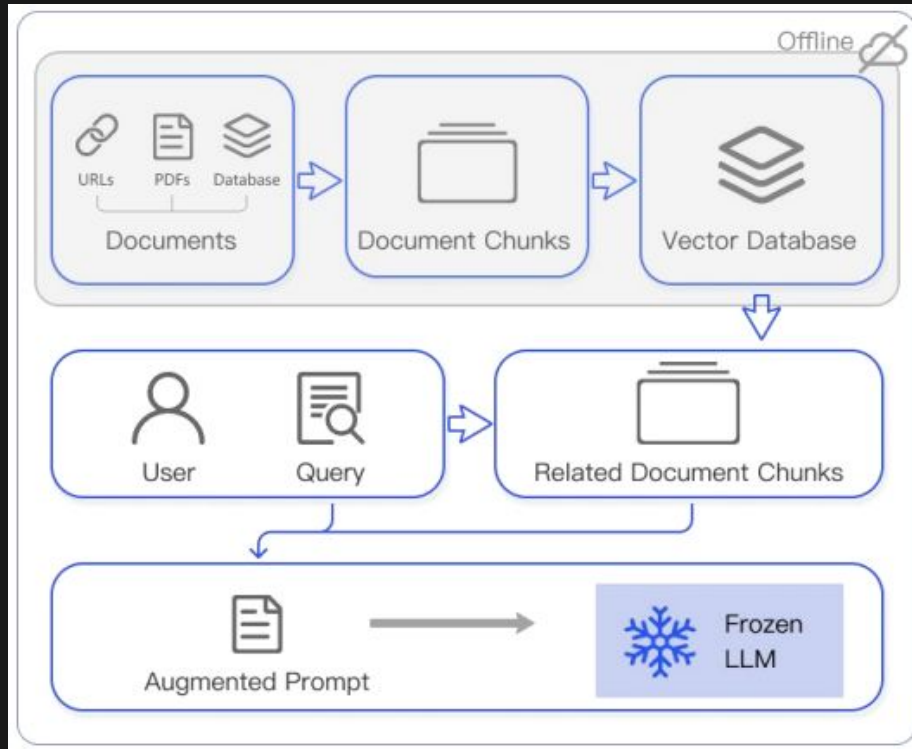
RAG lahendab kõik need probleemid:

-  Vastus põhineb tegelikel dokumentidel → vähem hallutsinatsioone
-  Dokumendid uuendatakse regulaarselt → info on ajakohane
-  Teie andmed jäävad teie serverisse → turvalisus säilib
-  Süsteem viitab allikale → vastus on kontrollitav

RAG – Retrieval-Augmented Generation

- **Samm 1 – Indekseerimine**
 - Jaga dokument ühtlasteks tükkideks, millest igaüks on osa algtekstist.
 - Kasuta keelemudelit, et luua iga tüki jaoks manustus (embedding).
 - Salvesta iga tüki embedding vektorandmebaasi.
- **Samm 2 – Otsimine**
 - Otsi vektorsarnasuse otsingu abil välja k kõige asjakohasemat dokumenti.
- **Samm 3 – Genereerimine**
 - Algne päring ja leitud tekstid ühendatakse ning saadetakse LLM-ile lõpliku vastuse saamiseks.

RAG – Retrieval-Augmented Generation



Task:

Respond to the user query using the provided context, incorporating inline citations in the format [id] ****only when the <source> tag includes an explicit id attribute**** (e.g., <source id="1">).

Guidelines:

- If you don't know the answer, clearly state that.
- If uncertain, ask the user for clarification.
- Respond in the same language as the user's query.
- If the context is unreadable or of poor quality, inform the user and provide the best possible answer.
- If the answer isn't present in the context but you possess the knowledge, explain this to the user and provide the answer using your own understanding.
- ****Only include inline citations using [id] (e.g., [1], [2]) when the <source> tag includes an id attribute.****
- Do not cite if the <source> tag does not contain an id attribute.
- Do not use XML tags in your response.
- Ensure citations are concise and directly related to the information provided.

Example of Citation:

If the user asks about a specific topic and the information is found in a source with a provided id attribute, the response should include the citation like in the following example:

* "According to the study, the proposed method increases efficiency by 20% [1]."

Output:

Provide a clear and direct response to the user's query, including inline citations in the format [id] **only when the <source> tag with id attribute is present in the context.**

```
<context>
{{CONTEXT}}
</context>
```

Kuidas RAG tekstitükke valib?

- Vector store-is on salvestatud tükide embeddingud (n qwen3 mudelist)
- Hübridotsing otsib valib välja tükide kandidaadid kahe kriteeriumi abil
 - BM25 - TF-IDF-laadne tekstipõhine meetod (in memory, LangChain)
 - Vektoriotsing - cosine similarity (ChromaDB)
- Ümberjärjestamine (reranking)
 - lokaalne CrossEncoder transformer
 - [CLS] <küsimus> [SEP] <tükk> [SEP]
 - [CLS] tokeni väljundvektori põhjal arvutatakse asjakohasuse skoor S_i
- Top-k valik
 - Lõplikku konteksti pääsevad ainult parimad K tükki.
 - Suurem K = rohkem konteksti, aga nõuab suuremat kontekstiakent

RAG tekstitükkide valimise seaded

Admin Panel → Settings → Documents

- Chunk Size – tüki suurus
- Chunk Overlap – tükkide kattumine
- Chunk Min Size Target – minimaalne tüki suurus
- Top K – valitavate tükkide arv
- Relevance Threshold – asjakohasuse lävend
- Embedding Model
- RAG Template – päringumall keelemudeli jaoks
- Markdown Header Splitting – päistepõhine tükeldus
- HYBRID_BM25_WEIGHT

Kuidas mõõdame RAG süsteemi kvaliteeti?

- Lähenemised
 - *LLM as a judge*
 - Manuaalne valideerimine
 - *Golden dataset*
- Kriteeriumid
 - Retrieval
 - Precision - kui suur osa leitud dokumentidest on päriselt asjakohased.
 - Recall - kui suure osa kõigist asjakohastest dokumentidest üles leiti
 - Mean reciprocal rank - kui kõrgele järjestab süsteem esimese õige dokumendi keskmiselt kõikide päringute lõikes
 - Generation
 - Faithfulness - kas vastus põhineb ainult leitud kontekstil, ilma väljamõeldud väideteta
 - Answer Relevance - kui hästi vastus tegelikult esitatud küsimusele vastab
- Tööriistad
 - RAGAS
 - DeepEval
 - TruLens

RAG vs fine-tuning?

Kasuta RAGi kui

- **Teadmistebaas peab olema ajakohane** – RAG toob kätte värskeid/uuendatud dokumente; peenhäälestus jäädvustab teadmised konkreetsetesse ajahetke
- **Vajad allikate viitamist** – RAG saab näidata, kust vastused pärinevad
- **Teadmistebaas on suur või dünaamiline** – tootedokumentatsioon, sisemised vikid, õigusandmebaasid, kliendiandmed
- **Soovid madalamat kulu/riski** – pole treeningtsükli, uuendamine on lihtne dokumentide vahetamisega
- **Hallutsinatsioonid on suur mure** – vastuste sidumine toodud tekstiga vähendab väljamõeldisi
- **Andmed on tundlikud** – võib-olla ei soovi sa ettevõttesiseseid andmeid mudeli kaaludesse põimida

Rusikareegel: **RAG muudab seda, mida mudel teab.**

RAG vs fine-tuning?

Kasuta *fine-tuningut* (*next token prediction objective*) kui

- **Vajad kindlat tooni, stiili või vormingut** — nt vasta alati lühidalt nagu CLI tööriist või brändi häälel
- **Soovid, et mudel õpiks teatud ülesandemustreid** — struktureeritud väljundivormingud, klassifitseerimisskeemid, valdkonnapõhised arutlusammud
- **Sul on märgistatud näited õigest käitumisest** — juhiste järgimine, koodistiil, vastuse ülesehitus
- **Latentsus on oluline** — ilma otsinguta vastused on kiiremad
- **Teadmised on stabiilsed ja põhimõttelised** — meditsiiniline terminoloogia, juriidiliste klauslite mustrid, kindel koodiraamistik

Rusikareegel: **Peenhäälestus muudab seda, kuidas mudel käitub.**

RAG + fine-tuning?

Peenhäälestatud mudel + RAG on võimas kombinatsioon:

- Peenhäälestus stiili, vormingu ja valdkonnapõhise arutluse jaoks
- RAG faktilise aluse ja ajakohase sisu jaoks

OpenWebUI arhitektuur – komponendid

- Web frontend SPA (SvelteKit)
 - Suhtleb backendiga, osalt üle WebSocketi
- Web backend (FastAPI)
 - Keskne orkestraator: teeb mudelitele päringuid, talletab andmeid
- Relational database (SQLite / PostgreSQL)
 - kasutajakontod, vestluste ajalugu, seaded, teadmiste metaandmed
- Vector Store (ChromaDB / etc)
- Cache (Redis)
 - WebSockets state ()
- Task queue (FastAPI's async event loop using asyncio)
- Model Server (Ollama)

OpenWebUI arhitektuur (1-node)

- open-webui konteiner
 - Web frontend (SvelteKit)
 - Web backend (FastAPI)
 - Relational database (SQLite)
 - Vector Store (in-memory ChromaDB)
 - Cache (in-memory)
 - Task queue (FastAPI's async event loop using asyncio)
- ollama konteiner
 - Model Server (Ollama)

 docker-compose.yml

▷ Run All Services

```
1  services:
    ▷ Run Service
2    ollama:
3      image: ollama/ollama:latest
4      volumes:
5        - ollama:/root/.ollama
6      ports:
7        - "11434:11434"
8      deploy:
9        resources:
10         reservations:
11         devices:
12           - driver: nvidia
13             count: all
14             capabilities: [gpu]
15
16     ▷ Run Service
17     open-webui:
18       image: ghcr.io/open-webui/open-webui:cuda
19       ports:
20         - "3000:8080"
21       volumes:
22         - open-webui:/app/backend/data
23       environment:
24         OLLAMA_BASE_URL: http://ollama:11434
25         USE_CUDA_DOCKER: "true"
26       deploy:
27         resources:
28         reservations:
29         devices:
30           - driver: nvidia
31             count: all
32             capabilities: [gpu]
33
34     volumes:
35       ollama: {}
36       open-webui: {}
```

Mis toimub, kui laen üles dokumendi?

1. Sisu eraldamine – toorfail läbib eraldamismootori. Tulemuseks on lihttekst.
2. Tükeldamine – OpenWebUI kasutab sisemiselt LangChaini tükeldajaid (tähemärgipõhised või markdown-teadlikud).
3. Teksti rikastamine – enne embeddingut lisatakse tükile dokumendi/sektsiooni pealkiri.
4. Tüki embedding – rikastatud tükk saadetakse embedding mudelile (Ollama)
5. Vector store lisamine - tagastatud tihe vektor salvestatakse vektoriandmebaasi.

1-node deploymenti puhul jooksevad sammud 1-3 ja 5 open-webui konteineris, samm 4 ollama konteineris.

Mis toimub, kui teen kasutajaliideses päringu?

1. Päringu embedimine (Ollama konteiner)
2. Vektori otsing, kas (open-webui konteiner)
 - a. Puhas vektorotsing (Vector Store)
 - b. Hübridotsing (Vector Store + in-memory BM25)
3. Ümberjärjestamine (valikuline), kas
 - a. open-webui konteineris CPU v GPU kaudu
 - i. pole verifitseerinud, kas meie setupil GPUd tegelikult kasutatakse
 - b. välise API kaudu (RAG_EXTERNAL_RERANKER_URL)
4. Top-K valimine ja konteksti lisamine ja vastuse genereerimine (Ollama)

```
1 services:
2   postgres: ...
3   redis: ...
20  >
34  > chromadb: ...
51  ollama-1:
52    image: ollama/ollama:latest
53    container_name: ollama-1
54    restart: unless-stopped
55    environment:
56      OLLAMA_HOST: 0.0.0.0
57      OLLAMA_KEEP_ALIVE: 24h
58      CUDA_VISIBLE_DEVICES: "0"
59    volumes:
60      - ollama_1_data:/root/.ollama
61    deploy:
62      resources:
63        reservations:
64          devices:
65            - driver: nvidia
66              device_ids: ["0"]
67              capabilities: [gpu]
68    networks:
69      - ai-net
70  > ollama-2: ...
89  > open-webui-1: ...
146 > open-webui-2: ...
196 > nginx: ...
209
210 volumes:
211   postgres_data:
212   redis_data:
213   chromadb_data:
214   ollama_1_data:
215   ollama_2_data:
216   open_webui_1_data:
217   open_webui_2_data:
218 |
219 networks:
220   ai-net:
```

Milline seadistus teie taristul?

1. H200 x4
 - a. variant 1
 - i. 2x ollama (GPU)
 - ii. 2x openweb-ui (GPU)
 - b. variant 2
 - i. 3x ollama (GPU)
 - ii. 1x openweb-ui (GPU)
 - c. variant 3
 - i. 3x ollama (GPU)
 - ii. 1x external reranker (GPU)
 - iii. ?x openweb-ui (CPU)
2. RTX PRO 6000 Blackwell x2
 - a. 1x ollama (GPU)
 - b. 1x openweb-ui (GPU)

Harjutus: süsteemi ülesseadmine

<https://github.com/mattiasarro/RAG>

qwen3.5:35b

Thought for 24 seconds ▾

NAFO (Never Again F**king Organisation) **looja Twitteris on ametlikult teadmata või anonüümne.**

Kuigi organisatsioonil on palju jälgijaid ja see on muutunud üheks kõige tunnustatumaks pro-Ukraina internettikultuuri liikumiseks Vene propoganda vastu võitlemisel, ei ole ükski konkreetne isik avalikult end loojana või juhtivna toimetajana esile toonud.

Siin on mõned peamised faktid NAFO kohta:

- **Looga:** Organisatsioon tekitas 2022. aasta algul, kohe pärast Venemaa suurt sissetungi Ukrainasse.
- **Olemus:** See on vabatahtlike ühendus (peamiselt ukraina aktivistid ja internetiliiklised), kes kasutavad meeme, ironiat ja trollimist Vene propoganda ümberlükkamiseks.
- **Tunnus:** Nad võtsid endale maskoti "NAFO" kutsariks (kelleks on tihti kujutatud hiirt või kassipoega).

Seega ei ole võimalik anda kindlat nime sellele Twitteri kasutajale, kes looja osaliselt alguse pani, kuna see on kavandatud kui kollektiivne anonüümne projekt.

Milline Twitteri kasutaja lõi NAFO?

qwen3.5:35b

Retrieved 3 sources

Thought for 26 seconds ▾

NAFO lõi 24. mail 2022 Twitteri kasutaja Kama_Kamilia NAFO (Interneti kogukond).md

3 Sources

Milliseid mudeleid valida?

1. Meil on vaja valida
 - a. *Generation model* (n qwen3.5:35b)
 - b. *Embedding model* (n qwen3-embedding)
 - c. *Reranking model* (n jinaai/jina-reranker-v2-base-multilingual)
2. Kriteeriumid
 - a. Eesti keele tugi
 - b. Mudeli kvaliteet (benchmark performance)
 - c. GPU piirangud
 - d. Open WebUI ühilduvus
 - i. Ollama mudelid: <https://ollama.com/library>
 - e. Kiirus

Valige nii suur ja kvaliteetne Eesti keelt võimaldav mudel kui GPUsse mahub.
Hiina mudelid on hetkel parimad; kinnises võrgus ohtu ei ole, aga poliitiline kalle eksisteerib.

Aitäh! 🙏